Guía de instalación y ejecución

Sistema multiagente urbanístico · NITEC

Objetivo
Dejar el sistema preparado en un ordenador nuevo y comprobar los requisitos básicos antes de ejecutarlo.

La guía sigue el orden recomendado: revisar el equipo, instalar Python y Ollama, preparar los archivos, instalar las dependencias y hacer una primera ejecución. Al final se incluye una lista de comprobaciones y soluciones para los errores más habituales.

Antes de empezar

- Ten a mano los archivos Python del sistema y los PDF de prueba.
- Necesitarás conexión a Internet para instalar Python, Ollama, las bibliotecas y los modelos.
- Los comandos están escritos para PowerShell en Windows. En otros sistemas, algunos pasos pueden variar.

Versión de trabajo: guía basada en la estructura actual del prototipo NITEC.

1. Requisitos del equipo

Antes de instalar nada, comprueba que el ordenador tiene recursos suficientes para ejecutar los modelos locales y guardar las bases de datos.

Elemento	Requisito
Sistema operativo	Windows 10/11 de 64 bits, macOS o Linux.
Memoria RAM	8 GB como mínimo. Se recomiendan 16 GB o más para trabajar con contextos amplios.
Espacio libre	Al menos 10 GB para Ollama, los modelos y las bases vectoriales.
Tarjeta gráfica	No es obligatoria. Una GPU compatible con CUDA o Apple Silicon puede acelerar el procesamiento; sin ella, Ollama puede usar la CPU.

2. Instalación desde cero

2.1. Instalar Python

Se recomienda Python 3.11 o 3.12. Son versiones adecuadas para las dependencias utilizadas por el prototipo y ayudan a evitar problemas de instalación en Windows.

Opción A: Miniconda

- Descarga el instalador desde la página oficial de Miniconda e instálalo con las opciones habituales.
- Abre Anaconda PowerShell Prompt o una terminal y comprueba la versión:

```
python --version
```

Opción B: instalador oficial de Python

- Descarga Python 3.11 o 3.12 desde python.org.
- Durante la instalación en Windows, marca **Add python.exe to PATH** antes de continuar.

2.2. Instalar Ollama y los modelos

NITEC utiliza Ollama para ejecutar los modelos de IA en el propio equipo. El prototipo necesita dos modelos: `llama3.1`, para las tareas de lenguaje, y `nomic-embed-text`, para convertir los documentos en vectores.

- Descarga e instala Ollama desde ollama.com/download.
- Comprueba que se ha instalado correctamente:

```
ollama --version
```

- Descarga los dos modelos. Ejecuta los comandos por separado y espera a que termine cada descarga:

```
ollama pull llama3.1
ollama pull nomic-embed-text
```

- Comprueba que aparecen en la lista:

```
ollama list
```

Deberían figurar `llama3.1:latest` y `nomic-embed-text:latest`.

2.3. Preparar la carpeta del proyecto

Crea una carpeta para NITEC y copia dentro los archivos del prototipo. Por ejemplo:

```
C:\NITEC_SOLUTIONS\Archivos_NITEC_Reales
```

Como mínimo, la carpeta debe incluir estos scripts:

- `main.py`

- `agente_1_extractor.py`
- `agente_2_normativo_local.py`
- `agente_3_procesador.py`
- `creador_base_datos.py`
- `creador_base_vectores.py`
- `geo_localizador.py`

Abre una terminal en esa carpeta. Puedes usar el comando `cd` y sustituir la ruta por la que hayas elegido:

```
cd "C:\NITEC_SOLUTIONS\Archivos_NITEC_Reales"
```

Si usas Conda, puedes crear un entorno independiente para este proyecto. Este paso es opcional, pero ayuda a mantener separadas sus bibliotecas:

```
conda create -n nitec python=3.11 -y
conda activate nitec
```

2.4. Instalar las bibliotecas de Python

Con la terminal situada en la carpeta del proyecto y el entorno correcto activado, instala las dependencias que usan los scripts:

```
pip install langchain langchain-core langchain-community langchain-text-splitters langchain-ollama langcha
```

Para qué se utilizan las principales bibliotecas:

- **LangChain** y sus paquetes relacionados: preparación de prompts, cadenas de ejecución y fragmentación de texto.
- **langchain-ollama**: conexión con los modelos locales de Ollama.
- **chromadb** y **langchain-chroma**: almacenamiento de las bases de datos vectoriales en disco.
- **pypdf**: lectura de PDF y operaciones de división de archivos.
- **rank-bm25**: búsqueda por palabras clave dentro del buscador híbrido del Agente 1.

Si el equipo ya tiene un archivo `requirements.txt` preparado para esta versión del proyecto, también se pueden instalar las dependencias con `pip install -r requirements.txt`.

3. Comprobaciones antes de ejecutar

Antes de lanzar `python main.py`, revisa estos puntos. Muchos de los errores de la primera ejecución se deben a que falta un modelo, un PDF válido o algún archivo de configuración.

1. Ollama está activo

En Windows, comprueba que Ollama está abierto en segundo plano. Si no lo está, abre la aplicación. También puedes iniciar el servicio desde una terminal con `ollama serve`, si procede.

2. Están descargados los dos modelos

Ejecuta `ollama list` y comprueba que aparecen `llama3.1` y `nomic-embed-text`.

3. Los PDF contienen texto seleccionable

Abre tanto el PDF del proyecto como el BOJA y prueba a seleccionar una frase. Si el documento es una imagen escaneada sin OCR, el cargador puede no extraer texto.

4. Revisa boja_base.json

Si es una instalación nueva, comprueba que no haya quedado un archivo antiguo, vacío o incompleto. El Agente 2 utiliza este archivo para conservar la información normativa que genera.

5. No ejecutes dos instancias a la vez

Evita tener dos terminales ejecutando `main.py` sobre la misma carpeta. Pueden competir por los archivos de las bases de datos.

6. Comprueba las rutas

Asegúrate de que las rutas introducidas son correctas y que la carpeta del proyecto contiene al menos un PDF. En Windows, el programa admite rutas con o sin comillas.

Nota sobre documentos escaneados: si el PDF no permite seleccionar texto, será necesario aplicarle OCR antes de usarlo con el flujo actual.

4. Archivos y carpetas que utiliza el sistema

Al ejecutar el prototipo, se crean o utilizan archivos auxiliares además de los scripts principales. La estructura puede quedar así:

```
Archivos_NITEC_Reales/
■■■ main.py
■■■ agente_1_extractor.py
■■■ agente_2_normativo_local.py
■■■ agente_3_procesador.py
■■■ creador_base_datos.py
■■■ creador_base_vectores.py
■■■ geo_localizador.py
■■■ municipios_espana.json
■■■ boja_base.json
■■■ normativas_ayuntamientos.json
■■■ normativas_pdf/
■■■ bd_normativas/
■■■ bd_vectores/
```

Qué guarda cada elemento:

- `main.py`: orquesta la ejecución y recoge los datos del proyecto.
- `agente_1_extractor.py`: busca la fase de tramitación y la fecha de aprobación.
- `agente_2_normativo_local.py`: genera o consulta `boja_base.json` y guarda los límites municipales.
- `agente_3_procesador.py`: comprueba el tamaño de los archivos y realiza la división de PDF prevista en el prototipo.
- `creador_base_datos.py`: crea la base vectorial de normativa en `bd_normativas`.
- `creador_base_vectores.py`: indexa los documentos del proyecto en `bd_vectores`.
- `geo_localizador.py` y `municipios_espana.json`: relacionan el municipio con su provincia y comunidad autónoma.
- `normativas_pdf/`: contiene los PDF normativos utilizados.
- `normativas_ayuntamientos.json`: conserva los límites de peso que se hayan registrado para los ayuntamientos.

La base `bd_vectores` corresponde al proyecto en curso. Según el comportamiento actual del prototipo, se reconstruye al procesar una nueva carpeta de proyecto.

5. Ejecutar el programa

Cuando las dependencias estén instaladas y las comprobaciones anteriores estén hechas, ejecuta el programa desde la carpeta del proyecto:

```
python main.py
```

El programa solicitará los datos del proyecto:

- **Municipio:** por ejemplo, Marbella o Málaga. El sistema intenta detectar la provincia y la comunidad autónoma.
- **Tipo de proyecto:** por ejemplo, ED, PGOM o POU.
- **Carpeta del proyecto:** ruta a la carpeta que contiene los PDF.
- **PDF principal:** puedes introducir la ruta del archivo o pulsar Enter para que el programa intente detectarlo.

En la primera ejecución puede ser necesario completar la configuración normativa:

- Si no existe `boja_base.json`, el Agente 2 analiza el PDF del BOJA y genera el archivo.
- Si el ayuntamiento todavía no está registrado, el programa puede pedir que introduzcas el límite de peso en MB. El dato se guarda en `normativas_ayuntamientos.json` para reutilizarlo en futuras ejecuciones.

Al terminar, el programa muestra un JSON consolidado con los datos geográficos, los metadatos que haya encontrado el Agente 1 y las reglas que haya podido recuperar el Agente 2. Que aparezca el JSON final no significa por sí solo que todos los campos se hayan extraído correctamente: revisa también los estados y mensajes de error.

6. Problemas habituales

Error	Posible causa	Qué hacer
ModuleNotFoundError: No module named 'rank_bm25'	No está instalada la biblioteca de búsqueda por palabras clave.	Ejecuta `pip install rank-bm25` en el entorno que utiliza el proyecto.
Fallo al conectar con localhost:11434	Ollama no está abierto o falta descargar un modelo.	Abre Ollama y comprueba los modelos con `ollama list`. Si hace falta, repite los comandos `ollama pull`.
El proyecto "XX" no existe en el BOJA	Puede haber quedado un archivo boja_base.json antiguo o la extracción no ha identificado el tipo de proyecto.	Revisa el contenido de `boja_base.json`. Si es un archivo de prueba obsoleto, guarda una copia y vuelve a generar el archivo.
Cambiar el límite municipal	El ayuntamiento ya tiene un valor guardado.	Edita el valor correspondiente en `normativas_ayuntamientos.json`. Si quieres que el programa vuelva a preguntarlo, haz una copia y elimina el archivo para regenerarlo.
El PDF no devuelve texto	El documento puede ser un escaneo sin capa de texto.	Aplica OCR al PDF y vuelve a probarlo.
El modelo devuelve texto en lugar de JSON	La respuesta del modelo no sigue el formato que espera el parser.	Muestra la respuesta bruta para diagnosticarla y comprueba el prompt y el formato de salida antes de confiar en los datos.

Recomendación: durante las pruebas, guarda la salida completa de la consola. Así podrás distinguir si ha fallado la conexión con Ollama, la lectura del PDF, la generación del JSON o la validación posterior de los datos.